

A
Semester Project–I Report
On
“Customer Churn Prediction System”

In partial fulfilment of requirements for the degree of

Bachelor of Technology
In
[Artificial Intelligence and Data Science.]

Submitted By

1. Dhiraj Sunil Mahajan (251108029 & AD026)
2. Mohit Shankar Zope (251108083 & AD069)
3. Hemant Sandip Fegade (251108090 & AD075)
4. Mayur Arun Mahajan (251108097 & AD081)

Under the Guidance of
Prof. Sandip S. Hire

[R. C. Patel Institute of Technology]
Shirpur, Dhule (Maharastra)
Autonomous
[Academic Year, 2026-27]

[R. C. Patel Institute of Technology]

Artificial Intelligence and Data Science

CERTIFICATE

This is to certify that the Semester Project–I entitled “AI-Based Fertilizer Recommendation System” has been carried out by the team:

1. Dhiraj Sunil Mahajan (251108029 & AD026)
2. Mohit Shankar Zope (251108083 & AD069)
3. Hemant Sandip Fegade (251108090 & AD075)
4. Mayur Arun Mahajan (251108097 & AD081)

under the guidance of [Guide’s Name] in partial fulfilment of the requirement for the degree of Bachelor of Technology in [Your Branch] (Semester–[N]) of [University Name] during the academic year [2026-27].

Date: [12/09/2026]

Place: Shirpur, Dhule.

[Prof. Sandip S. Hire]

[Dr. D. R. Patil]

[Prof. Sandip S. Hire]

[Dr. J. B. Patil]

Guide

H.O.D.

Semester Project–I Co-Ordinator

Director/Principal

Abstract

Fertilizer use in Indian agriculture is largely guided by habit, local dealer advice, or generic per-crop dosage charts, rather than by the specific nutrient status of a farmer's own field. This mismatch between what a field actually needs and what is applied leads to two recurring problems: excess fertilizer that raises cost and degrades soil health over time, and insufficient fertilizer that caps yield below what the field could otherwise support. Government soil health cards address part of this gap, but they are issued infrequently and do not adapt to variation in temperature, humidity, or rainfall across a growing season.

This project proposes a web-based Fertilizer Recommendation System that uses a trained machine learning model to suggest a fertilizer type and approximate quantity for a given field. The system takes soil nutrient readings (nitrogen, phosphorus, potassium), soil pH, and local climate data (temperature, humidity, rainfall) along with the crop being grown, and returns a recommendation through a simple web interface that does not require the user to understand the underlying model.

During Stage-I, the project focuses on problem identification, a survey of existing machinelearning approaches to fertilizer and crop recommendation, requirement analysis, system architecture, database design, and the design and frontend implementation of the user interface.

Model training and full backend integration are planned for Stage-II.

Contents

1 Introduction	4
1.1 Background.....	4
1.2 Problem Statement	4
1.3 Objectives	5
1.4 Scope of the Project	5
1.4.1 Existing System	6
1.4.2 Limitations of Existing System	6
2 Literature Survey.....	7
2.1 Machine Learning in Precision Agriculture	7
2.2 Fertilizer and Crop Recommendation Datasets.....	7

2.3 Ensemble Methods for Structured Agricultural Data	7
2.4 Comparative Study	7
2.5 Identified Research Gap.....	8
3 System Analysis and Requirements	9
3.1 Feasibility Study	9
3.1.1 Technical Feasibility.....	9
3.1.2 Operational Feasibility	9
3.1.3 Economic Feasibility.....	9
3.1.4 Functional Requirements	9
3.1.5 Non-Functional Requirements.....	10
4 System Design and Architecture	10
4.1 System Architecture	10
4.1.1 Presentation Layer.....	11
4.1.2 Application and Logic Layer	11
4.1.3 Data Persistence Layer	11
4.2 Data Flow Analysis	11
4.2.1 Level 0: Context-Level Flow.....	11
4.2.2 Level 1: Functional Sub-Processes	11
4.2.3 Level 2: Process Decomposition	12
4.3 Database Schema Design.....	12
4.3.1 Data Dictionary: tbl query.....	12
4.3.2 Data Dictionary: tbl fertilizer.....	12
5 System Modeling and Implementation Plan	13
5.1 Use Case Specifications.....	13
5.1.1 User Interactions	13
5.1.2 Administrator Interactions	13
5.2 Class-Level Design.....	14
5.3 Interaction Flow	14
5.4 State Lifecycle of a Query	15

5.5 Algorithmic Flow of Recommendation Generation	15
6 Expected Results and Testing Strategy	16
6.1 Expected Results	16
6.2 Software Testing Strategy	16
6.2.1 Unit Testing	16
6.2.2 Integration and Functional Test Cases	16
6.2.3 Security Testing	16
6.3 Future Scope	17
Conclusion	17

1 Introduction

1.1 Background

Agriculture remains central to the Indian economy, and the productivity of a field depends heavily on whether it receives the right nutrients in the right amounts. Fertilizer is one of the few inputs a farmer can control directly, yet the decision of how much and what type to apply is often made without reference to an actual soil test. A farmer typically either follows what was used the previous season, adopts a neighbour's practice, or takes the recommendation of the local fertilizer dealer, whose interest is naturally tied to what is available in stock rather than what the soil specifically requires.

Machine learning offers a practical way to close this gap. Given a reasonably sized dataset that maps soil and climate readings to known-good fertilizer choices for different crops, a classification model can be trained to generalize that mapping and produce a recommendation for a new, unseen combination of readings. This project applies that idea in the form of an accessible web tool.

1.2 Problem Statement

Existing fertilizer decisions on the ground suffer from the following issues:

1. Lack of personalization: Recommendations are usually based on a generic per-crop ratio rather than the farmer's actual soil test values.
2. Infrequent testing: Government soil health cards are issued once every two to three years and do not reflect current-season conditions.
3. Over- and under-application: Without a clear reference, farmers tend to either overapply fertilizer out of caution, which increases cost and can harm soil structure over time, or under-apply it, which limits yield.
4. No accessible digital tool: Most existing agricultural advisory tools focus on crop selection or pest identification; very few combine soil nutrient values, climate data, and crop type into a single fertilizer-specific recommendation that a farmer or agricultural officer can use directly.

1.3 Objectives

- To design a web-based system where a user can enter soil nutrient values (N, P, K), pH, and climate readings (temperature, humidity, rainfall) along with the crop being grown.
- To use a trained classification model to recommend an appropriate fertilizer type and an approximate quantity range for the given input.
- To keep the interface simple enough that it can be used by an agricultural extension officer on behalf of a farmer, without requiring any understanding of machine learning.
- To store past queries so that recommendations for a given field can be reviewed over time.
- To validate the design against a real fertilizer/crop dataset before moving to full model integration in Stage-II.

1.4 Scope of the Project

The system is intended for individual farmers, agricultural extension workers, and Krishi Vigyan Kendra staff who need a quick, data-backed second opinion on fertilizer choice. Stage-I of this project covers requirement analysis, dataset study, system architecture, database planning, and the design and implementation of the user-facing interface. Stage-II will cover training and validating the classification model, connecting it to the interface through a backend service, and testing the complete system end to end.

1.4.1 Existing System

Existing practice combines three sources: the government soil health card scheme, informal advice from input dealers, and standard per-crop fertilizer charts published by agricultural universities. Some newer mobile apps provide general crop advisory content, but fertilizer-specific, soil-test-driven recommendation is rarely part of these tools in a form a small farmer can use directly.

1.4.2 Limitations of Existing System

- Soil health card data quickly becomes outdated as a season progresses.
- Dealer advice is not necessarily aligned with the farmer's actual soil condition.
- Generic fertilizer charts do not account for the specific N-P-K balance already present in a given field.
- No lightweight, freely accessible web tool combines soil, climate, and crop data into one recommendation step.

2 Literature Survey

2.1 Machine Learning in Precision Agriculture

Precision agriculture research over the last decade has increasingly used machine learning to turn soil and climate data into actionable advice, most commonly for crop selection and yield prediction. Classification algorithms such as Decision Trees, Random Forests, and Support Vector Machines have been applied to datasets that record soil nutrient values against a known best-fit crop or treatment, generally with good accuracy on held-out test data.

2.2 Fertilizer and Crop Recommendation Datasets

Several publicly available datasets pair soil nutrient readings (N, P, K), pH, temperature, humidity, and rainfall with a labelled outcome, either a recommended crop or a recommended fertilizer. These datasets are commonly used to benchmark classification approaches for agricultural recommendation systems, and form the basis for the model this project intends to train in Stage-II.

2.3 Ensemble Methods for Structured Agricultural Data

Among classification approaches, ensemble methods such as Random Forest tend to perform well on structured tabular data of the kind used in fertilizer recommendation, since they handle the mix of continuous inputs (like pH and rainfall) and categorical inputs (like crop type) without needing heavy preprocessing, and are less prone to overfitting on smaller agricultural datasets than a single decision tree.

2.4 Comparative Study

Table 2.1 compares the approach used in this project against the two dominant existing approaches.

Table 2.1: Comparison of Fertilizer Guidance Approaches

Parameter	Generic Chart	Fertilizer Soil Health Card	Proposed System
-----------	------------------	--------------------------------	-----------------

Personalization	None	Low (outdated by season)	High (live input)
Update quency	Fre- Static	Once every 2–3 years	Every query
Accessibility	Print/offline	Government office	Any web browser
Basis for Advice	Crop only	One-time test	N, P, K, pH, climate, crop

2.5 Identified Research Gap

Academic work in this space tends to focus on maximizing model accuracy on a fixed dataset, with comparatively little attention paid to how such a model would actually be used by a farmer in the field. There is a gap for a simple, low-cost web tool that takes a trained classification model and wraps it in an interface usable by someone with no technical background, which is the direction this project takes.

3 System Analysis and Requirements

3.1 Feasibility Study

3.1.1 Technical Feasibility

The system can be built entirely with open-source tools: Python with scikit-learn for training and running the classification model, Flask as a lightweight backend framework, and a relational database such as MySQL or SQLite for storing users and query history. None of these require specialized hardware.

3.1.2 Operational Feasibility

The end user only needs to fill in a form with numeric soil and climate values and select a crop from a dropdown; no training is required. An agricultural extension officer can enter this data on a farmer's behalf during a field visit.

3.1.3 Economic Feasibility

All planned tools and libraries are open source, so the only ongoing cost is hosting. There is no licensing fee for the machine learning framework, the backend framework, or the database.

Table 3.1: Hardware Requirements

Component	Type	Minimum Specification
Server	Processor	Dual-core Intel i3 / AMD Ryzen 3 or higher
Server	Memory	4 GB RAM
Server	Storage	10 GB available space
Client	Device	Any smartphone, tablet or desktop with a browser

3.1.4 Functional Requirements

- FR-01: The system shall accept soil nutrient values (N, P, K), pH, temperature, humidity, and rainfall as numeric input.

Table 3.2: Software Requirements

Layer	Technology
Frontend	HTML5, CSS3, JavaScript

Backend	Python (Flask)
Model Training	scikit-learn, pandas, numpy
Database	MySQL / SQLite
Version Control	Git and GitHub

- FR-02: The system shall accept a crop selection from a predefined list.
- FR-03: The system shall validate all inputs against sensible agronomic ranges before submission.
- FR-04: The system shall return a recommended fertilizer type and an approximate quantity range.
- FR-05: The system shall store each query along with its recommendation for later review.
- FR-06: The system shall allow an administrator to update the reference table of fertilizers and their typical composition.

3.1.5 Non-Functional Requirements

- Performance: A recommendation should be returned within 2 seconds of submission.
- Usability: The interface should be usable on a mobile browser without horizontal scrolling.
- Reliability: Invalid or out-of-range input should be rejected with a clear message rather than causing an error.
- Maintainability: The reference data for fertilizers and crops should be editable without changing code.

4 System Design and Architecture

4.1 System Architecture

The system follows a three-tier architecture consisting of a presentation layer, an application layer, and a data layer.

4.1.1 Presentation Layer

This is the web interface where a user enters soil and climate readings and selects a crop, and where the recommendation is displayed once generated. It is designed mobile-first so it remains usable on the kind of device most likely to be available in the field.

4.1.2 Application and Logic Layer

This layer validates incoming input, prepares it in the format the trained model expects, calls the model to generate a prediction, and maps the model's output class to a fertilizer name and a typical quantity range drawn from a reference table.

4.1.3 Data Persistence Layer

A relational database stores user accounts (if login is enabled), the history of submitted queries and their recommendations, and reference tables for fertilizers and crops.

4.2 Data Flow Analysis

4.2.1 Level 0: Context-Level Flow

At the top level, the system interacts with two external entities: the User (farmer or extension officer), who submits soil and climate data and receives a recommendation, and the Administrator, who maintains the fertilizer and crop reference data.

4.2.2 Level 1: Functional Sub-Processes

- Process 1.0 – Input Collection: Captures soil, climate, and crop data from the user through the web form.
- Process 2.0 – Input Validation: Checks that all values fall within realistic agronomic ranges.
- Process 3.0 – Model Inference: Passes the validated, formatted input to the trained classification model and receives a predicted fertilizer class.
- Process 4.0 – Recommendation Formatting: Maps the predicted class to a fertilizer name, composition, and quantity range using the reference table.
- Process 5.0 – Query Logging: Stores the submitted input and the resulting recommendation against a timestamp.

4.2.3 Level 2: Process Decomposition

Within Process 2.0 and Process 3.0:

- Sub-process 2.1: Confirms each numeric field is present and within its expected range (for example, pH between 0 and 14).
- Sub-process 2.2: Confirms the selected crop matches one of the crops the model was trained on.
- Sub-process 3.1: Encodes the crop as a categorical feature and assembles the full feature vector.
- Sub-process 3.2: Passes the feature vector to the loaded model object and retrieves the predicted class along with a confidence score.

4.3 Database Schema Design

The schema is designed to Third Normal Form to avoid redundant storage of fertilizer and crop details across query records.

4.3.1 Data Dictionary: tbl query

4.3.2 Data Dictionary: tbl fertilizer

Table 4.1: Database Entities

Entity	Primary Key	Core Attributes
tbl user	user id	name, email, password hash, role
tbl query	query id	user fk, nitrogen, phosphorus, potassium, ph, temperature, humidity, rainfall, crop, submitted at
tbl fertilizer	fertilizer id	name, composition, typical dose min, typical dose max
tbl recommendation	recommendation id	query fk, fertilizer fk, confidence score

Field	Type	Null	Description
query id	INT(11)	No	Auto-increment primary key
user fk	INT(11)	Yes	References tbl user(user id)
nitrogen	FLOAT	No	Nitrogen reading in kg/ha
phosphorus	FLOAT	No	Phosphorus reading in kg/ha

potassium	FLOAT	No	Potassium reading in kg/ha
ph	FLOAT	No	Soil pH value
temperature	FLOAT	No	Temperature in C
humidity	FLOAT	No	Relative humidity in %
rainfall	FLOAT	No	Rainfall in mm
crop	VARCHAR(50)	No	Selected crop name
submitted at	DATETIME	No	Query timestamp
Field	Type	Null	Description
fertilizer id	INT(11)	No	Auto-increment primary key
name	VARCHAR(80)	No	Fertilizer name (e.g., Urea, DAP)
composition	VARCHAR(100)	No	N-P-K composition ratio
typical dose min	FLOAT	No	Lower bound of typical dose (kg/ha)
typical dose max	FLOAT	No	Upper bound of typical dose (kg/ha)

5 System Modeling and Implementation Plan

5.1 Use Case Specifications

The system supports two roles: the User (farmer or extension officer) and the Administrator.

5.1.1 User Interactions

- Enter soil, climate, and crop details through the web form.
- Submit the form and view the resulting fertilizer recommendation.
- View past queries and their recommendations, if logged in.

5.1.2 Administrator Interactions

- Add, edit, or remove entries in the fertilizer reference table.
- Review submitted queries for data quality checks.

- Trigger retraining of the model once new labelled data becomes available (planned for Stage-II).

5.2 Class-Level Design

- InputValidator: Checks each field against its expected range and type before it reaches the model.
- FeatureEncoder: Converts validated input, including the categorical crop field, into the numeric feature vector the model expects.
- RecommendationService: Loads the trained model, runs inference, and maps the output class to a fertilizer record.
- QueryRepository: Handles reading and writing query and recommendation records to the database.
- AdminController: Handles reference-table updates and basic authentication for the administrator role.

5.3 Interaction Flow

1. The user fills in the soil, climate, and crop form and submits it.
2. The browser sends the form data to the backend as an HTTP request.
3. InputValidator checks the data; if any field fails validation, an error message is returned immediately.
4. FeatureEncoder converts the validated data into a feature vector.
5. RecommendationService passes the vector to the trained model and receives a predicted fertilizer class.
6. The predicted class is matched against tblfertilizer to retrieve the fertilizer name, composition, and dose range.
7. QueryRepository stores the query and the recommendation.
8. The recommendation is returned to the browser and displayed to the user.

5.4 State Lifecycle of a Query

- Submitted: Raw form data has been received by the server.
- Validated: All fields have passed range and type checks.
- Predicted: The model has returned a fertilizer class for the given input.
- Logged: The query and recommendation have been saved to the database.
- Reviewed (optional): An administrator has checked the query, typically as part of data quality review before using it in future model retraining.

5.5 Algorithmic Flow of Recommendation Generation

1. Input extraction: Read nitrogen, phosphorus, potassium, pH, temperature, humidity, rainfall, and crop from the request.
2. Validation: Confirm each value lies within its expected agronomic range; reject the request otherwise.
3. Encoding: Represent the crop as a categorical feature consistent with how the model was trained, and assemble the full feature vector $X = [N, P, K, pH, T, H, R, \text{crop}]$.
4. Inference: Pass X to the trained classifier to obtain a predicted fertilizer class $\hat{y}$.
5. Lookup: Retrieve the fertilizer name, composition, and dose range associated with $\hat{y}$ from `tblfertilizer`.
6. Response assembly: Combine the fertilizer details into a response object for the frontend.
7. Logging: Store the input, the predicted class, and the response in `tblquery` and `tblrecommendation`.

6 Expected Results and Testing Strategy

6.1 Expected Results

- A working web form that accepts soil, climate, and crop data and validates it before submission.
- A clear, consistent format for displaying a recommendation once the model is connected in Stage-II.
- A database schema capable of logging every query, which can later be used as additional training data.
- A system architecture and interface design that a non-technical user, such as a farmer or extension officer, can operate without assistance.

6.2 Software Testing Strategy

6.2.1 Unit Testing

- Verify that InputValidator correctly rejects out-of-range values (for example, pH above 14 or negative rainfall).
- Verify that FeatureEncoder produces a feature vector of the expected length and order for every supported crop.
- Verify that RecommendationService returns a valid fertilizer ID for a known input once the model is integrated.

6.2.2 Integration and Functional Test Cases

6.2.3 Security Testing

- Use parameterized queries to prevent SQL injection through form fields.
- Sanitize and escape any user-entered text before it is rendered back on the page.

Table 6.1: Planned Functional Test Cases

Test ID	Module	Input Condition	Expected Output	Status
TC-01	Validation	All fields within range	Request accepted	Planned
TC-02	Validation	pH value of 20	Rejected with range error	Planned
TC-03	Validation	Missing crop field	Rejected with missing-field error	Planned
TC-04	Inference	Valid input for Rice	Fertilizer class returned	Planned
TC-05	Logging	Successful recommendation	Query and result saved to DB	Planned
TC-06	Admin	Update fertilizer dose range	Reference table reflects new values	Planned

- Rate-limit form submissions to reduce the risk of automated abuse once the system is deployed publicly.

6.3 Future Scope

- Train and integrate the classification model using a labelled fertilizer/crop dataset, completing the Stage-II backend.
- Add support for regional languages so the tool is usable by a wider range of farmers.
- Explore integration with government soil health card data to pre-fill known values for a given field.
- Investigate a simple SMS or IVR-based interface for users without reliable smartphone access.

Conclusion

This report has covered the motivation, requirement analysis, architecture, database design, and interface design for an AI-based fertilizer recommendation system. The core idea is straightforward: replace guesswork-driven fertilizer decisions with a recommendation grounded in the field's actual soil and climate readings, delivered through an interface simple enough for a farmer or extension officer to use directly.

Stage-I has established the problem definition, a survey of related machine-learning approaches, the system's three-tier architecture, a normalized database schema, and a working frontend for data entry and result display. Stage-II will build on this foundation by training the classification model on a labelled dataset, connecting it to the interface through the planned Flask backend, and testing the complete pipeline from data entry through to a live recommendation.

Bibliography

- [1] I. Sommerville, *Software Engineering*, 10th ed. Pearson Education, 2016.
- [2] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [3] Food and Agriculture Organization of the United Nations, "Fertilizer Use by Crop," FAO, Rome, 2006.
- [4] Pallets Projects, "Flask Documentation," 2024. [Online]. Available: <https://flask.palletsprojects.com/>
- [5] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [6] OWASP Foundation, "OWASP Top Ten Web Application Security Risks," 2023. [Online]. Available: <https://owasp.org/www-project-top-ten/>